

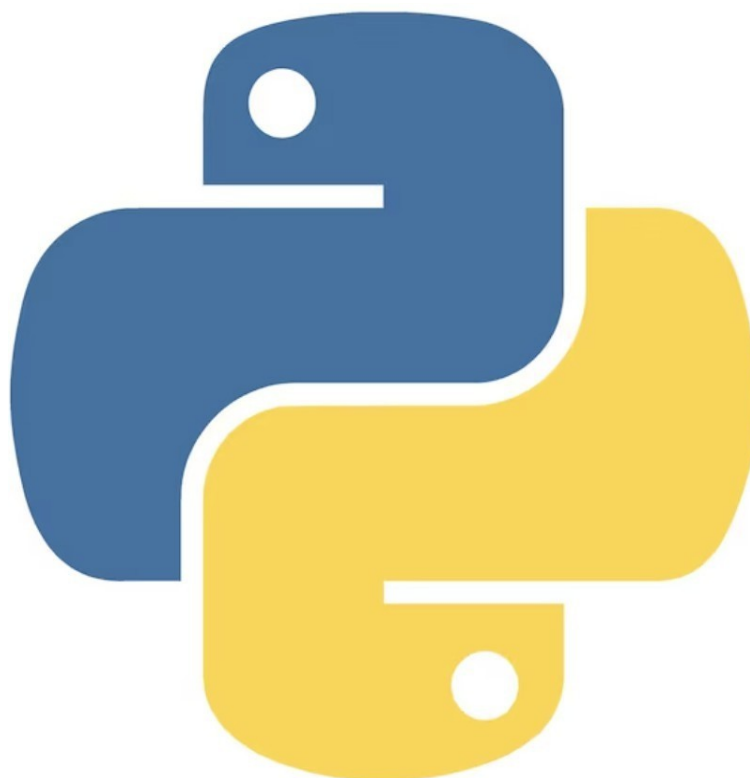


INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL, I.P.
CENTRO DE EMPREGO E FORMAÇÃO PROFISSIONAL DE BRAGA

EFA – PROGRAMADOR/A DE INFORMÁTICA

UFCD 10794

Python Avançado



Lucas Gonçalves, 08

Trabalho sobre: Plataforma Web de Python

Animals Clínica Veterinária.

Formador: Marcos Alvarães



Índice

Resumo do Projeto.....	3
Descrição das Funcionalidades do Sistema.....	4
Construção da Dados: Criação da Tabela Cliente e User.....	4
Construção da Dados: Criação da Tabela Cliente e User.....	5
Função: ligar_bd, imports e Criação da Classe Construtora Flask.....	6
Função: Bloco de Funções.....	7
Função: Rota Index.....	8
Função: users_listar.....	15
Função: criar_cliente.....	19
Função: criar_user.....	20
Função: editar_user.....	23
Função: editar_cliente.....	24
Função: editar_animal.....	25
Função: editar_consulta.....	26
Função: apagar_cliente.....	27
Função: apagar_user.....	29
Função: apagar_consulta.....	30
Função: apagar_cliente.....	31



Resumo do Projeto

Sistema da Animals Clínica Veterinária

Este projeto consiste no desenvolvimento de um sistema web para gestão de uma clínica veterinária, com o objetivo de facilitar o controle de utilizadores, clientes, animais e consultas, centralizando todas as informações em uma única plataforma.

O sistema foi desenvolvido utilizando o framework Flask (Python), seguindo o padrão MVC (Model–View–Controller), onde:

- o **backend** é responsável pelo processamento das regras de negócio,
- o **frontend** apresenta as interfaces ao utilizador através de templates HTML,
- o **banco de dados MySQL** armazena todas as informações do sistema.

A aplicação possui um sistema de **autenticação e controle de acesso**, permitindo que apenas utilizadores autenticados acessem áreas restritas. Os utilizadores são organizados por **perfis (admin, staff e cliente)**, onde cada perfil possui permissões específicas dentro do sistema.

- Entre as principais funcionalidades do sistema destacam-se:
- Registo e autenticação de utilizadores
- Gestão de clientes
- Cadastro e listagem de animais associados a cada cliente
- Registo e consulta de consultas veterinárias
- Área exclusiva do cliente (“Minha Área”), onde é possível visualizar seus dados, seus animais e suas consultas
- Proteção de rotas utilizando sessões para garantir a segurança da aplicação

O projeto tem como foco **organização, segurança e usabilidade**, permitindo que a clínica tenha um controle eficiente das informações, reduzindo erros manuais e melhorando o atendimento aos clientes.



Descrição das Funcionalidades do Sistema

Sistema da Animals Clínica Veterinária

Construção da Dados: Criação da Tabela Cliente e User

Descrição Simplificada: Realiza a criação de uma tabela no MySQL Workbench e armazena a descrição na estrutura da tabela Clientes e a estrutura da tabela User, a inserção dos dados foi feita no próprio programa de Python na construção do projeto.

Mostra a tabela Clientes e Users :

```
CREATE DATABASE IF NOT EXISTS efa0125_08_vet_clinic;
USE efa0125_08_vet_clinic;

-- Clientes (donos dos animais)
CREATE TABLE clientes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(120) NOT NULL,
  telefone VARCHAR(30),
  email VARCHAR(120) UNIQUE,
  morada VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Utilizadores do sistema (admin/staff/cliente)
-- cliente_id é usado apenas quando role='cliente'
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  role ENUM('admin','staff','cliente') NOT NULL,
  cliente_id INT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_users_cliente
    FOREIGN KEY (cliente_id) REFERENCES clientes(id)
    ON DELETE SET NULL
);
```



Construção da Dados: Criação da Tabela Cliente e User

Descrição Simplificada: Realiza a criação de uma tabela no MySQL Workbench e armazena a descrição na estrutura da tabela Cliente e a estrutura da Tabela User, a inserção dos dados foi feita no próprio programa de Python na construção do projeto.

Mostra a tabela Animais e Consultas:

```
CREATE DATABASE IF NOT EXISTS efa0125_08_vet_clinic;
USE efa0125_08_vet_clinic;

-- Clientes (donos dos animais)
CREATE TABLE clientes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(120) NOT NULL,
  telefone VARCHAR(30),
  email VARCHAR(120) UNIQUE,
  morada VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Utilizadores do sistema (admin/staff/cliente)
-- cliente_id é usado apenas quando role='cliente'
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  role ENUM('admin','staff','cliente') NOT NULL,
  cliente_id INT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_users_cliente
    FOREIGN KEY (cliente_id) REFERENCES clientes(id)
    ON DELETE SET NULL
);
```



Função: **ligar_bd, imports e Criação da Classe Construtora Flask**

Descrição simplificada: Estabelece a conexão com o MySQL, importa as bibliotecas essenciais do Flask e inicializa a aplicação com uma chave de sessão.

Mostrar a função:

```
from flask import render_template, request, redirect, url_for, session, flash
import json
import requests
import datetime

# Função que cria a ligação com o banco de dados MySQL
def ligar_db():
    return mysql.connector.connect(
        host="62.28.39.135",      # Endereço do servidor MySQL
        user="efa0125",          # Nome do utilizador da base de dados
        password="123.Abc",      # Palavra-passe da base de dados
        database="efa0125_08_vet_clinic" # Nome da base de dados
    )

##### CRIAÇÃO DA APLICAÇÃO FLASK #####
# Cria a aplicação Flask
app = Flask(__name__)

# Chave secreta usada para sessões (necessária para login)
app.secret_key = "123"
```



Função: **Bloco de Funções**

Descrição Simplificada: Bloco com as principais funções usadas no programa. Referente ao principais papeis na sessão definida pelo programa e usado para permissão de admin, staff ou cliente ao longo da sessão.

Mostra a função:

```
# =====  
# Funções de permissões  
# =====  
  
def admin():  
    # Retorna True se o utilizador tiver o papel "admin"  
    return session.get("role") == "admin"  
  
def staff():  
    # Retorna True se o utilizador tiver o papel "staff"  
    return session.get("role") == "staff"  
  
def is_cliente():  
    # Retorna True se o utilizador tiver o papel "cliente"  
    return session.get("role") == "cliente"  
  
# =====  
# Disponibilizar funções no Jinja  
# =====  
  
# Esta função diz ao Flask para disponibilizar variáveis e funções  
# automaticamente em TODOS os templates HTML  
@app.context_processor  
def inject_roles():  
    # Torna as funções admin(), staff() e cliente() acessíveis no HTML  
    return dict(  
        admin=admin,  
        staff=staff,  
        is_cliente=is_cliente  
    )  
  
def datetime_ano():  
    # Retorna o ano atual para usar no footer  
    return {"ano": datetime.datetime.now().year}
```




Função: **Rota Index**

Descrição simplificada: Exibe a página inicial do sistema e redireciona utilizadores autenticados para o dashboard.

Mostra da função:

```
# =====  
# ROTA PRINCIPAL (INDEX)  
# =====  
@app.route("/")  
def index():  
  
    # Se o utilizador já estiver autenticado,  
    # redireciona para o dashboard.  
    if "user_id" in session:  
        return redirect(url_for("dashboard"))  
  
    # Se NÃO estiver autenticado,  
    # mostra a página inicial pública.  
    return render_template("index.html")
```




Função: Rota de login

Descrição simplificada: Realiza a autenticação dos utilizadores, validando credenciais e criando a sessão de acesso.

Mostra da função:

```
@app.route("/login", methods=["GET", "POST"])
def login():
    # Se já existe sessão ativa, evita novo login e redireciona para o dashboard
    if "user_id" in session:
        return redirect(url_for("dashboard"))

    # GET → apenas mostra o formulário de login
    if request.method == "GET":
        return render_template("login.html")

    # POST → recolhe os dados enviados pelo formulário
    username = request.form.get("username", "").strip()
    password = request.form.get("password", "").strip()

    # Verifica se os campos obrigatórios foram preenchidos
    if not username or not password:
        flash("Preencha todos os campos.")
        return redirect(url_for("login"))

    conexao = ligar_db()
    cursor = conexao.cursor(dictionary=True)

    try:
        # Procura o utilizador com username e password correspondentes
        cursor.execute("""
            SELECT id, username, password, role, cliente_id
            FROM users
            WHERE username = %s AND password = %s
        """, (username, password))

        user = cursor.fetchone()

        # Se não encontrou utilizador, as credenciais são inválidas
        if not user:
            flash("Credenciais inválidas.")
            return redirect(url_for("login"))

        # LOGIN OK → limpa sessão antiga e cria nova
        session.clear()
        session["user_id"] = user["id"]
        session["role"] = user["role"]
        session["username"] = user["username"]
```



Função: **Rota de login**

Descrição simplificada: Autentica utilizadores, valida permissões e cria a sessão conforme o tipo de utilizador.

Mostra da função:

```
# Se o utilizador for cliente, deve ter cliente_id associado
if user["role"] == "cliente":
    if not user["cliente_id"]:
        flash("Erro: este utilizador cliente não está associado a um cliente.")
        return redirect(url_for("login"))

    # Guarda o cliente_id na sessão para filtrar dados do cliente
    session["cliente_id"] = user["cliente_id"]

flash("Login efetuado com sucesso!")
return redirect(url_for("dashboard"))

except Exception as erro:
    # Regista o erro e mostra mensagem genérica
    app.logger.exception("ERRO AO EFETUAR LOGIN")
    flash("Erro ao efetuar login. Tente novamente.")
    return redirect(url_for("login"))

finally:
    # Fecha a ligação ao banco
    cursor.close()
    conexao.close()

# =====
```



Função: Rota de logout e Inicializacao do Flask

Descrição Simplificada: Encerra a sessão do utilizador e inicia a aplicação Flask em modo de desenvolvimento.

Mostra da função:

```
# =====  
# LOGOUT  
# =====  
@app.route("/logout")  
def logout():  
    # Limpa todos os dados da sessão, terminando a autenticação do utilizador  
    session.clear()  
  
    # Redireciona o utilizador para a página de login após sair  
    return redirect(url_for("login"))  
  
# Iniciar a aplicação Flask  
if __name__ == "__main__":  
    app.run(debug=True)
```



Função: **Rota minha_conta, meus_animais e minhas_consulta (parte 1)**

Descrição simplificada e mais específica: Controlam o acesso à área do cliente, verificando sessão e permissões, obtendo da base de dados os seus dados pessoais, os animais registados e as consultas marcadas, enviando tudo para os respetivos templates.

Mostra da função:

```
# =====
# MINHA CONTA (apenas cliente)
# =====
@app.route("/minha_conta")
def minha_conta():
    # Verifica se existe sessão ativa; caso contrário, redireciona para login
    if "user_id" not in session:
        return redirect(url_for("login"))

    # Apenas clientes podem aceder a esta página
    if session.get("role") != "cliente":
        flash("Acesso negado. Apenas clientes podem ver esta página.")
        return redirect(url_for("dashboard"))

    conexao = ligar_db()
    cursor = conexao.cursor(dictionary=True)

    try:
        # Busca os dados do cliente associado ao utilizador autenticado
        cursor.execute("SELECT * FROM clientes WHERE id = %s", (session.get("cliente_id"),))
        cliente = cursor.fetchone()

        # Renderiza a página com os dados do cliente
        return render_template("minha_conta.html", cliente=cliente)

    finally:
        # Fecha a ligação ao banco
        cursor.close()
        conexao.close()
```



Função: **Rota minha_conta, meus_animais e minhas_consultas (parte 2)**

Descrição simplificada e específica: Controlam o acesso à área do cliente, verificando sessão e permissões, obtendo da base de dados os seus dados pessoais, os animais registados e as consultas marcadas, enviando tudo para os respetivos templates.

Mostra da função:

```
# =====
# MEUS ANIMAIS (apenas cliente)
# =====
@app.route("/meus_animais")
def meus_animais():
    # Verifica se existe sessão ativa; caso contrário, redireciona para login
    if "user_id" not in session:
        return redirect(url_for("login"))

    # Apenas clientes podem aceder a esta página
    if session.get("role") != "cliente":
        flash("Apenas clientes podem ver os seus próprios animais.")
        return redirect(url_for("dashboard"))

    conexao = ligar_db()
    cursor = conexao.cursor(dictionary=True)

    try:
        # Busca todos os animais pertencentes ao cliente autenticado
        cursor.execute("""
            SELECT id, nome, especie, raca, data_nascimento
            FROM animais
            WHERE cliente_id = %s
            ORDER BY nome
            """, (session.get("cliente_id"),))

        animais = cursor.fetchall()

        # Renderiza a página com a lista de animais
        return render_template("meus_animais.html", animais=animais)

    finally:
        # Fecha a ligação ao banco
        cursor.close()
        conexao.close()
```



Função: **Rota minha_conta, meus_animais e minhas_consultas (parte 3)**

Descrição simplificada e específica: Garante o acesso apenas a clientes autenticados, controlam o acesso à área do cliente, verificando sessão e permissões, obtendo da base de dados os seus dados pessoais, os animais registados e as consultas marcadas, enviando tudo para os respetivos templates.

Mostra da função:

```
# =====
# MINHAS CONSULTAS (apenas cliente)
# =====
@app.route("/minhas_consultas")
def minhas_consultas():
    # Verifica se existe sessão ativa; caso contrário, redireciona para login
    if "user_id" not in session:
        return redirect(url_for("login"))

    # Apenas clientes podem aceder a esta página
    if session.get("role") != "cliente":
        flash("Apenas clientes podem ver as suas consultas.")
        return redirect(url_for("dashboard"))

    conexao = ligar_db()
    cursor = conexao.cursor(dictionary=True)

    try:
        # Busca todas as consultas dos animais pertencentes ao cliente autenticado
        cursor.execute("""
            SELECT
                consultas.id,
                animais.nome AS animal,
                consultas.data_hora,
                consultas.motivo,
                consultas.notas,
                consultas.created_at
            FROM consultas
            INNER JOIN animais ON consultas.animal_id = animais.id
            WHERE animais.cliente_id = %s
            ORDER BY consultas.data_hora DESC
        """, (session.get("cliente_id"),))

        consultas = cursor.fetchall()

        # Renderiza a página com a lista de consultas
        return render_template("minhas_consultas.html", consultas=consultas)

    finally:
        # Fecha a ligação ao banco
        cursor.close()
        conexao.close()
```



Função: `users_listar`

Descrição simplificada: Lista todos os utilizadores do sistema, permitindo acesso apenas a administradores ou staff, obtendo os dados na base de dados e enviando-os para o template.

Mostra da função:

```
# =====
# LISTAR USERS (apenas admin)
# =====
@app.route("/users")
def users_listar():
    # Apenas administradores e staff podem ver a lista completa de utilizadores
    if not (admin() or staff()):
        flash("Não tem permissão para ver utilizadores.")
        return redirect(url_for("dashboard"))

    # Liga à base de dados
    conexao = ligar_db()
    cursor = conexao.cursor(dictionary=True)

    # Busca todos os utilizadores e junta o nome do cliente associado (se existir)
    cursor.execute("""
        SELECT users.*, clientes.nome AS cliente_nome
        FROM users
        LEFT JOIN clientes ON clientes.id = users.cliente_id
        ORDER BY users.username
    """)
    users = cursor.fetchall()

    # Fecha a ligação
    cursor.close()
    conexao.close()

    # Renderiza o template com a lista de utilizadores
    return render_template("users_listar.html", users=users)
```




Função: **clientes_listar**

Descrição simplificada: Lista todos os clientes do sistema, permitindo acesso apenas a administradores ou staff, obtendo os dados na base de dados e enviando-os para o template.

Mostra da função:

```
# =====  
# LISTAR CLIENTES (admin e staff)  
# =====  
@app.route("/clientes")  
def clientes_listar():  
    # Apenas admin e staff podem ver a lista completa de clientes  
    if not (admin() or staff()):  
        flash("Não tem permissão para ver clientes.")  
        return redirect(url_for("dashboard"))  
  
    # Liga à base de dados  
    conexao = ligar_db()  
    cursor = conexao.cursor(dictionary=True)  
  
    # Busca todos os clientes ordenados por nome  
    cursor.execute("SELECT * FROM clientes ORDER BY nome")  
    clientes = cursor.fetchall()  
  
    # Fecha a ligação  
    cursor.close()  
    conexao.close()  
  
    # Renderiza o template com a lista de clientes  
    return render_template("clientes_listar.html", clientes=clientes)
```



Função: **animais_listar**

Descrição simplificada: Lista todos os animais registados no sistema, permitindo acesso apenas a administradores ou staff, obtendo os dados na base de dados juntamente com o nome do cliente dono e enviando-os para o template.

Mostra da função:

```
# =====  
# LISTAR ANIMAIS  
# =====  
@app.route("/animais")  
def animais_listar():  
    # Apenas admin e staff podem ver todos os animais  
    if not (admin() or staff()):  
        flash("Não tem permissão para ver animais.")  
        return redirect(url_for("dashboard"))  
  
    # Liga à base de dados  
    conexao = ligar_db()  
    cursor = conexao.cursor(dictionary=True)  
  
    # Busca todos os animais e junta o nome do cliente dono  
    cursor.execute("""  
        SELECT animais.*, clientes.nome AS cliente_nome  
        FROM animais  
        INNER JOIN clientes ON clientes.id = animais.cliente_id  
        ORDER BY animais.nome  
    """)  
    animais = cursor.fetchall()  
  
    # Fecha a ligação  
    cursor.close()  
    conexao.close()  
  
    # Renderiza o template com a lista de animais  
    return render_template("animais_listar.html", animais=animais)
```



Função: [consultas_listar](#)

Descrição simplificada: Lista todas as consultas registadas no sistema, permitindo acesso apenas a administradores ou staff, obtendo os dados completos das consultas juntamente com o nome do animal e do cliente, e enviando tudo para o template.

Mostra da função:

```
# =====
# LISTAR CONSULTAS
# =====
@app.route("/consultas")
def consultas_listar():
    # Apenas admin e staff podem ver todas as consultas
    if not (admin() or staff()):
        flash("Não tem permissão para ver consultas.")
        return redirect(url_for("dashboard"))

    # Liga à base de dados
    conexao = ligar_db()
    cursor = conexao.cursor(dictionary=True)

    # Busca todas as consultas, juntando o nome do animal e do cliente
    cursor.execute("""
        SELECT consultas.*,
               animais.nome AS animal_nome,
               clientes.nome AS cliente_nome
        FROM consultas
        INNER JOIN animais ON animais.id = consultas.animal_id
        INNER JOIN clientes ON clientes.id = animais.cliente_id
        ORDER BY consultas.data_hora DESC
    """)
    consultas = cursor.fetchall()

    # Fecha ligação
    cursor.close()
    conexao.close()

    # Renderiza o template
    return render_template("consultas_listar.html", consultas=consultas)
```



Função: criar_cliente

Descrição simplificada: Insere um novo **cliente** na tabela *clientes*, verificando permissões, recolhendo os dados enviados pelo formulário, validando-os e gravando o registo na base de dados.

Mostra da função:

```
# =====
# CRIAR CLIENTE (admin e staff)
# =====
@app.route("/criar_cliente", methods=["GET", "POST"])
def criar_cliente():
    # Apenas admin e staff podem criar clientes
    if not (admin() or staff()):
        flash("Não tem permissão para criar clientes.")
        return redirect(url_for("dashboard"))

    # Liga à base de dados
    conexao = ligar_db()
    cursor = conexao.cursor(dictionary=True)

    try:
        # POST real (cliquou no botão Criar)
        if request.method == "POST" and request.form.get("acao") == "salvar":

            # Recolhe os dados enviados pelo formulário
            nome = request.form.get("nome", "").strip()
            telefone = (variable) request: Request, "").strip()
            email = request.form.get("email", "").strip()
            morada = request.form.get("morada", "").strip()

            # Validação simples
            if not nome:
                flash("O nome é obrigatório.")
                return render_template("criar_cliente.html")

            # Insere o cliente na base de dados
            cursor.execute("""
                INSERT INTO clientes (nome, telefone, email, morada)
                VALUES (%s, %s, %s, %s)
            """, (nome, telefone, email, morada))

            conexao.commit()
            flash("Cliente criado com sucesso!")
            return redirect(url_for("clientes_listar"))

        # GET normal → mostra o formulário
        return render_template("criar_cliente.html")

    except Exception:
        app.logger.exception("ERRO AO CRIAR CLIENTE")
        conexao.rollback()
        flash("Erro ao criar cliente.")
        return redirect(url_for("clientes_listar"))

    finally:
        cursor.close()
        conexao.close()
```



Função: `criar_user`

Descrição simplificada: Insere um novo **utilizador** na tabela *users*, verificando permissões, carregando a lista de clientes, recolhendo dados do formulário, validando-os e inserindo o registo na base de dados.

Mostra da função:

```
# =====
# CRIAR UTILIZADOR (apenas admin)
# =====
@app.route("/criar_user", methods=["GET", "POST"])
def criar_user():
    # Apenas administradores podem criar utilizadores
    if not admin():
        flash("Apenas administradores podem criar utilizadores.")
        return redirect(url_for("dashboard"))

    # Liga à base de dados
    conexao = ligar_db()
    cursor = conexao.cursor(dictionary=True)

    # Carrega lista de clientes para o dropdown
    cursor.execute("SELECT id, nome FROM clientes ORDER BY nome")
    clientes = cursor.fetchall()

    try:
        # POST real (clicou no botão Criar)
        if request.method == "POST" and request.form.get("acao") == "salvar":

            # Recolhe os dados enviados pelo formulário
            username = request.form.get("username", "").strip()
            password = request.form.get("password", "").strip()
            role_form = request.form.get("role", "").strip()

            # Trata o cliente_id (pode vir vazio)
            cliente_id_raw = request.form.get("cliente_id", "").strip()
            cliente_id = int(cliente_id_raw) if cliente_id_raw.isdigit() else None

            # Validação simples
            if not username or not password:
                flash("Username e password são obrigatórios.")
                return render_template("criar_user.html", clientes=clientes)

            # Insere o novo utilizador
            cursor.execute("""
                INSERT INTO users (username, password, role, cliente_id)
                VALUES (%s, %s, %s, %s)
            """, (username, password, role_form, cliente_id))

            conexao.commit()
            flash("Utilizador criado com sucesso!")
            return redirect(url_for("users_listar"))

        # GET normal → mostra o formulário
        return render_template("criar_user.html", clientes=clientes)

    finally:
        cursor.close()
        conexao.close()
```



Função: `criar_animal`

Descrição Simplificada: Insere um novo **animal** na tabela *animais*, verificando permissões, carregando clientes para o formulário, recolhendo os dados enviados e inserindo-os na base de dados após validação.

Mostra da função:

```
# =====
# CRIAR ANIMAL (admin, staff e cliente)
# =====
@app.route("/criar_animal", methods=["GET", "POST"])
def criar_animal():
    # Apenas admin e staff podem criar animais
    if not (admin() or staff()):
        flash("Não tem permissão para criar animais.")
        return redirect(url_for("dashboard"))

    # Liga à base de dados
    conexao = ligar_db()
    cursor = conexao.cursor(dictionary=True)

    # Carrega lista de clientes para o dropdown
    cursor.execute("SELECT id, nome FROM clientes ORDER BY nome")
    clientes = cursor.fetchall()

    try:
        # POST real (clizou no botão Criar)
        if request.method == "POST" and request.form.get("acao") == "salvar":

            nome = request.form.get("nome", "").strip()
            especie = request.form.get("especie", "").strip()
            raca = request.form.get("raca", "").strip()
            data_nascimento = request.form.get("data_nascimento", "").strip()

            cliente_id_raw = request.form.get("cliente_id", "").strip()
            cliente_id = int(cliente_id_raw) if cliente_id_raw.isdigit() else None

            # Validação simples
            if not nome or not especie or not cliente_id:
                flash("Nome, espécie e cliente são obrigatórios.")
                return render_template("criar_animal.html", clientes=clientes)

            # Insere o animal
            cursor.execute("""
                INSERT INTO animais (cliente_id, nome, especie, raca, data_nascimento)
                VALUES (%s, %s, %s, %s, %s)
            """, (cliente_id, nome, especie, raca, data_nascimento or None))

            conexao.commit()
            flash("Animal criado com sucesso!")
            return redirect(url_for("animais_listar"))

        # GET normal
        return render_template("criar_animal.html", clientes=clientes)

    finally:
        cursor.close()
        conexao.close()
```



Função: **criar_consulta**

Descrição Simplificada: Insere uma nova **consulta** na tabela *consultas*, verificando permissões, carregando a lista de animais, recolhendo os dados do formulário, validando-os e gravando o registo na base de dados.

Mostra da função:

```
# CRIAR CONSULTA (admin, staff e cliente)
# =====
@app.route("/criar_consulta", methods=["GET", "POST"])
def criar_consulta():
    # Apenas admin e staff podem criar consultas
    if not (admin() or staff()):
        flash("Não tem permissão para criar consultas.")
        return redirect(url_for("dashboard"))

    conexao = ligar_db()
    cursor = conexao.cursor(dictionary=True)

    # Carrega lista de animais (com nome do cliente)
    cursor.execute("""
        SELECT animais.id, animais.nome, clientes.nome AS cliente_nome
        FROM animais
        INNER JOIN clientes ON clientes.id = animais.cliente_id
        ORDER BY animais.nome
    """)
    animais = cursor.fetchall()

    try:
        if request.method == "POST" and request.form.get("acao") == "salvar":

            animal_id_raw = request.form.get("animal_id", "").strip()
            animal_id = int(animal_id_raw) if animal_id_raw.isdigit() else None

            data_hora = request.form.get("data_hora", "").strip()
            motivo = request.form.get("motivo", "").strip()
            notas = request.form.get("notas", "").strip()

            # Validação simples
            if not animal_id or not data_hora:
                flash("Animal e data/hora são obrigatórios.")
                return render_template("criar_consulta.html", animais=animais)

            # Insere a consulta
            cursor.execute("""
                INSERT INTO consultas (animal_id, data_hora, motivo, notas)
                VALUES (%s, %s, %s, %s)
            """, (animal_id, data_hora, motivo, notas))

            conexao.commit()
            flash("Consulta criada com sucesso!")
            return redirect(url_for("consultas_listar"))

        return render_template("criar_consulta.html", animais=animais)

    finally:
        cursor.close()
        conexao.close()

    # =====
    # GET ou POST AUTOMÁTICO
    # =====
    return render_template("criar_consulta.html", clientes=clientes, animais=animais)
```




Função: `editar_user`

Descrição Simplificada: Edita os dados de um utilizador existente, verificando permissões, carregando o utilizador e a lista de clientes, recolhendo os dados enviados pelo formulário, validando-os e atualizando o registo na base de dados.

Mostra da função:

```
@app.route("/editar_user/<int:id>", methods=["GET", "POST"])
def editar_user(id):
    # Apenas admin e staff podem editar utilizadores
    if not (admin() or staff()):
        flash("Não tem permissão para editar utilizadores.")
        return redirect(url_for("dashboard"))

    conexao = ligar_db()
    cursor = conexao.cursor(dictionary=True)

    # Busca o utilizador pelo ID
    cursor.execute("SELECT * FROM users WHERE id = %s", (id,))
    user = cursor.fetchone()

    if not user:
        flash("Utilizador não encontrado.")
        return redirect(url_for("users_listar"))

    # Carrega lista de clientes
    cursor.execute("SELECT id, nome FROM clientes ORDER BY nome")
    clientes = cursor.fetchall()

    try:
        if request.method == "POST" and request.form.get("acao") == "salvar":
            username = request.form.get("username", "").strip()
            role_form = request.form.get("role", "").strip()

            cliente_id_raw = request.form.get("cliente_id", "").strip()
            cliente_id = int(cliente_id_raw) if cliente_id_raw.isdigit() else None

            # Apenas admin pode alterar password
            nova_password = None
            if admin():
                nova_password_raw = request.form.get("password", "").strip()
                if nova_password_raw:
                    nova_password = generate_password_hash(nova_password_raw)

            if not username:
                flash("Username é obrigatório.")
                return render_template("editar_user.html",
                                       user=request.form,
                                       clientes=clientes)

            # Atualiza o utilizador
            if nova_password:
                cursor.execute("""
                    UPDATE users
                    SET username=%s, role=%s, cliente_id=%s, password=%s
                    WHERE id=%s
                    """, (username, role_form, cliente_id, nova_password, id))
            else:
                cursor.execute("""
                    UPDATE users
                    SET username=%s, role=%s, cliente_id=%s
                    WHERE id=%s
                    """, (username, role_form, cliente_id, id))

            conexao.commit()
            flash("Utilizador atualizado com sucesso!")
            return redirect(url_for("users_listar"))

        return render_template("editar_user.html",
                               user=user,
                               clientes=clientes)

    finally:
        cursor.close()
        conexao.close()
```




Função: editar_cliente

Descrição Simplificada: Edita os dados de um cliente existente no sistema, verificando permissões, carregando o cliente pelo ID, recolhendo os dados enviados pelo formulário, validando-os e atualizando o registo na base de dados.

Mostra da função:

```
# EDITAR CLIENTE (admin e staff)
# =====
@app.route("/editar_cliente/<int:id>", methods=["GET", "POST"])
def editar_cliente(id):
    # Apenas admin e staff podem editar clientes
    if not (admin() or staff()):
        flash("Não tem permissão para editar clientes.")
        return redirect(url_for("dashboard"))

    # Liga à base de dados
    conexao = ligar_db()
    cursor = conexao.cursor(dictionary=True)

    # Busca o cliente pelo ID
    cursor.execute("SELECT * FROM clientes WHERE id = %s", (id,))
    cliente = cursor.fetchone()

    if not cliente:
        flash("Cliente não encontrado.")
        return redirect(url_for("clientes_listar"))

    try:
        # POST real (clicou em Guardar)
        if request.method == "POST" and request.form.get("acao") == "salvar":

            nome = request.form.get("nome", "").strip()
            telefone = request.form.get("telefone", "").strip()
            email = request.form.get("email", "").strip()
            morada = request.form.get("morada", "").strip()

            # Validação simples
            if not nome:
                flash("O nome é obrigatório.")
                return render_template("editar_cliente.html", cliente=request.form)

            # Atualiza o cliente
            cursor.execute("""
                UPDATE clientes
                SET nome=%s, telefone=%s, email=%s, morada=%s
                WHERE id=%s
            """, (nome, telefone, email, morada, id))

            conexao.commit()
            flash("Cliente atualizado com sucesso!")
            return redirect(url_for("clientes_listar"))

        # GET normal
        return render_template("editar_cliente.html", cliente=cliente)

    except Exception:
        app.logger.exception("ERRO AO EDITAR CLIENTE")
        conexao.rollback()
        flash("Erro ao atualizar cliente.")
        return redirect(url_for("clientes_listar"))

    finally:
        cursor.close()
        conexao.close()
```



Função: editar_animal

Descrição Simplificada: Edita os dados de um animal existente no sistema, verificando permissões, carregando o animal e a lista de clientes, recolhendo os dados enviados pelo formulário, validando-os e atualizando o registo na base de dados.

Mostra da função:

```
# =====
# EDITAR ANIMAL (admin, staff e cliente)
# =====
@app.route("/editar_animal/<int:id>", methods=["GET", "POST"])
def editar_animal(id):
    # Apenas admin e staff podem editar animais
    if not (admin() or staff()):
        flash("Não tem permissão para editar animais.")
        return redirect(url_for("dashboard"))

    conexao = ligar_db()
    cursor = conexao.cursor(dictionary=True)

    # Busca o animal pelo ID
    cursor.execute("SELECT * FROM animais WHERE id = %s", (id,))
    animal = cursor.fetchone()

    if not animal:
        flash("Animal não encontrado.")
        return redirect(url_for("animais_listar"))

    # Carrega lista de clientes
    cursor.execute("SELECT id, nome FROM clientes ORDER BY nome")
    clientes = cursor.fetchall()

    try:
        if request.method == "POST" and request.form.get("acao") == "salvar":

            nome = request.form.get("nome", "").strip()
            especie = request.form.get("especie", "").strip()
            raca = request.form.get("raca", "").strip()
            data_nascimento = request.form.get("data_nascimento", "").strip()

            cliente_id_raw = request.form.get("cliente_id", "").strip()
            cliente_id = int(cliente_id_raw) if cliente_id_raw.isdigit() else None

            if not nome or not especie or not cliente_id:
                flash("Nome, espécie e cliente são obrigatórios.")
                return render_template("editar_animal.html",
                                       animal=request.form,
                                       clientes=clientes)

            cursor.execute("""
                UPDATE animais
                SET cliente_id=%s, nome=%s, especie=%s, raca=%s, data_nascimento=%s
                WHERE id=%s
            """, (cliente_id, nome, especie, raca, data_nascimento or None, id))

            conexao.commit()
            flash("Animal atualizado com sucesso!")
            return redirect(url_for("animais_listar"))

        return render_template("editar_animal.html",
                               animal=animal,
                               clientes=clientes)

    finally:
        cursor.close()
        conexao.close()
```



Função: editar_consulta

Descrição Simplificada: Edita uma consulta existente no sistema, verificando permissões, carregando a consulta e a lista de animais, recolhendo os dados enviados pelo formulário, validando-os e atualizando o registo na base de dados.

Mostra da função:

```
def editar_consulta(id):
    # Apenas admin e staff podem editar consultas
    if not (admin() or staff()):
        flash("Não tem permissão para editar consultas.")
        return redirect(url_for("dashboard"))

    conexao = ligar_db()
    cursor = conexao.cursor(dictionary=True)

    # Busca a consulta pelo ID
    cursor.execute("SELECT * FROM consultas WHERE id = %s", (id,))
    consulta = cursor.fetchone()

    if not consulta:
        flash("Consulta não encontrada.")
        return redirect(url_for("consultas_listar"))

    # Carrega lista de animais
    cursor.execute("""
        SELECT animais.id, animais.nome, clientes.nome AS cliente_nome
        FROM animais
        INNER JOIN clientes ON clientes.id = animais.cliente_id
        ORDER BY animais.nome
    """)
    animais = cursor.fetchall()

    try:
        if request.method == "POST" and request.form.get("acao") == "salvar":

            animal_id_raw = request.form.get("animal_id", "").strip()
            animal_id = int(animal_id_raw) if animal_id_raw.isdigit() else None

            data_hora = request.form.get("data_hora", "").strip()
            motivo = request.form.get("motivo", "").strip()
            notas = request.form.get("notas", "").strip()

            if not animal_id or not data_hora:
                flash("Animal e data/hora são obrigatórios.")
                return render_template("editar_consulta.html",
                                       consulta=request.form,
                                       animais=animais)

            cursor.execute("""
                UPDATE consultas
                SET animal_id=%s, data_hora=%s, motivo=%s, notas=%s
                WHERE id=%s
            """, (animal_id, data_hora, motivo, notas, id))

            conexao.commit()
            flash("Consulta atualizada com sucesso!")
            return redirect(url_for("consultas_listar"))

        return render_template("editar_consulta.html",
                               consulta=consulta,
                               animais=animais)

    finally:
        cursor.close()
        conexao.close()
```



Função: [apagar_cliente](#)

Descrição: Remove um cliente do sistema, verificando permissões, apagando o registo na base de dados e redirecionando o utilizador com uma mensagem de confirmação.

Mostra da função:

```
# =====
# APAGAR CLIENTE (apenas admin)
# =====
@app.route("/apagar_cliente/<int:id>", methods=["POST"])
def apagar_cliente(id):
    # Apenas administradores podem apagar clientes
    if not admin():
        flash("Apenas administradores podem apagar clientes.")
        return redirect(url_for("dashboard"))

    conexao = ligar_db()
    cursor = conexao.cursor()

    # Apaga o cliente pelo ID
    cursor.execute("DELETE FROM clientes WHERE id = %s", (id,))
    conexao.commit()

    cursor.close()
    conexao.close()

    flash("Cliente apagado com sucesso!")
    return redirect(url_for("clientes_listar"))

# =====
# APAGAR ANIMAL (apenas admin)
# =====
@app.route("/apagar_animal/<int:id>", methods=["POST"])
def apagar_animal(id):
    # Apenas administradores podem apagar animais
    if not admin():
        flash("Apenas administradores podem apagar animais.")
        return redirect(url_for("dashboard"))

    conexao = ligar_db()
    cursor = conexao.cursor()

    cursor.execute("DELETE FROM animais WHERE id = %s", (id,))
    conexao.commit()

    cursor.close()
    conexao.close()

    flash("Animal apagado com sucesso!")
    return redirect(url_for("animais_listar"))
```



Função: `apagar_user`

Descrição Simplificada: Remove um utilizador do sistema, verificando permissões, apagando o registo correspondente na base de dados e redirecionando o utilizador com uma mensagem de confirmação.

Mostra da função:

```
# =====
# APAGAR UTILIZADOR (apenas admin)
# =====
@app.route("/apagar_user/<int:id>", methods=["POST"])
def apagar_user(id):
    if not admin():
        flash("Apenas administradores podem apagar utilizadores.")
        return redirect(url_for("dashboard"))

    conexao = ligar_db()
    cursor = conexao.cursor()

    cursor.execute("DELETE FROM users WHERE id = %s", (id,))
    conexao.commit()

    cursor.close()
    conexao.close()

    flash("Utilizador apagado com sucesso!")
    return redirect(url_for("users_listar"))

# =====
# LISTAR CLIENTES (admin e staff)
# =====
@app.route("/clientes")
def clientes_listar():
    # Apenas admin e staff podem ver a lista completa de clientes
    if not (admin() or staff()):
        flash("Não tem permissão para ver clientes.")
        return redirect(url_for("dashboard"))

    # Liga à base de dados
    conexao = ligar_db()
    cursor = conexao.cursor(dictionary=True)

    # Busca todos os clientes ordenados por nome
    cursor.execute("SELECT * FROM clientes ORDER BY nome")
    clientes = cursor.fetchall()

    # Fecha a ligação
    cursor.close()
    conexao.close()

    # Renderiza o template com a lista de clientes
    return render_template("clientes_listar.html", clientes=clientes)
```



Função: **apagar_consulta**

Descrição Simplificada: Remove uma consulta do sistema, verificando sessão ativa, permissões de administrador, executando a remoção na base de dados e informando o utilizador do resultado da operação.

Mostra da função:

```
# =====
# APAGAR CONSULTA (apenas admin)
# =====
@app.route("/apagar_consulta/<int:id>", methods=["POST"])
def apagar_consulta(id):
    # Verifica se existe sessão ativa; caso contrário, redireciona para login
    if "user_id" not in session:
        return redirect(url_for("login"))

    # Apenas administradores podem apagar consultas
    if not admin():
        flash("Apenas administradores podem apagar registos.")
        return redirect(url_for("dashboard"))

    conexao = ligar_db()
    cursor = conexao.cursor(dictionary=True)

    try:
        # Tenta apagar a consulta pelo ID
        cursor.execute("DELETE FROM consultas WHERE id = %s", (id,))
        conexao.commit()
        flash("Consulta apagada com sucesso!")

    except Exception as erro:
        # Regista o erro e desfaz a operação
        app.logger.exception("ERRO AO APAGAR CONSULTA")
        conexao.rollback()
        flash("Erro ao apagar consulta. Verifique dependências ou tente novamente.")

    finally:
        # Fecha a ligação ao banco
        cursor.close()
        conexao.close()

    # Redireciona após tentativa de apagar
    return redirect(url_for("consultas_listar"))
```




Função: [apagar_cliente](#)

Descrição Simplificada: Remove um cliente do sistema, garantindo que apenas administradores podem executar esta ação e eliminando o registo correspondente na base de dados.

Mostra da função:

```
# =====
# APAGAR CLIENTE (apenas admin)
# =====
@app.route("/apagar_cliente/<int:id>", methods=["POST"])
def apagar_cliente(id):
    # Apenas administradores podem apagar clientes
    if not admin():
        flash("Apenas administradores podem apagar clientes.")
        return redirect(url_for("dashboard"))

    conexao = ligar_db()
    cursor = conexao.cursor()

    # Apaga o cliente pelo ID
    cursor.execute("DELETE FROM clientes WHERE id = %s", (id,))
    conexao.commit()

    cursor.close()
    conexao.close()

    flash("Cliente apagado com sucesso!")
    return redirect(url_for("clientes_listar"))

# =====
# APAGAR ANIMAL (apenas admin)
# =====
@app.route("/apagar_animal/<int:id>", methods=["POST"])
def apagar_animal(id):
    # Apenas administradores podem apagar animais
    if not admin():
        flash("Apenas administradores podem apagar animais.")
        return redirect(url_for("dashboard"))

    conexao = ligar_db()
    cursor = conexao.cursor()

    cursor.execute("DELETE FROM animais WHERE id = %s", (id,))
    conexao.commit()

    cursor.close()
    conexao.close()

    flash("Animal apagado com sucesso!")
    return redirect(url_for("animais_listar"))
```



Função: **trocar_password**

Descrição: Altera a senha de um utilizador, garantindo que apenas o próprio utilizador ou um administrador pode realizar a alteração, validando os dados e atualizando a password na base de dados.

Mostra da função:

```
@app.route("/trocar_password/<int:id>", methods=["GET", "POST"])
def trocar_password(id):
    # Verifica se está logado
    if "user_id" not in session:
        return redirect(url_for("login"))

    # Apenas o próprio utilizador ou admin podem alterar
    if session["user_id"] != id and not admin():
        flash("Não tem permissão para alterar esta password.")
        return redirect(url_for("dashboard"))

    conexao = ligar_db()
    cursor = conexao.cursor(dictionary=True)

    # Buscar dados do utilizador
    cursor.execute("SELECT id, username FROM users WHERE id=%s", (id,))
    user = cursor.fetchone()

    if not user:
        flash("Utilizador não encontrado.")
        return redirect(url_for("dashboard"))

    if request.method == "POST":
        nova = request.form.get("nova_password", "").strip()
        confirmar = request.form.get("confirmar_password", "").strip()

        # Validação
        if not nova or not confirmar:
            flash("Preencha todos os campos.")
            return render_template("users/trocar_password.html", user=user)

        if nova != confirmar:
            flash("As passwords não coincidem.")
            return render_template("users/trocar_password.html", user=user)

        # Atualizar password
        cursor.execute("""
            UPDATE users SET password=%s WHERE id=%s
            """, (nova, id))

        conexao.commit()
        cursor.close()
        conexao.close()

        flash("Password alterada com sucesso!")
        return redirect(url_for("dashboard"))

    cursor.close()
    conexao.close()

    return render_template("users/trocar_password.html", user=user)
```